

תרגיל בית 1 –

אוטומט תאי

Authors: Kobie Hazon and Daniel Ben Zion

קבצי ההגשה:

בשל הגודל של קובץ ההרצה (שכן הוא קובץ פייתון שקומפל – ולכן מכיל את כל רכיבי ההרצה של פייתון 3.8), הקבצים נמצאים בתיקיית גוגל דרייב שהקישור אליה מופיע [כאן](#).

התיקייה מכילה את הקבצים והתיקיות הבאות:

- תיקיית src - מכילה את הקוד של סימולטור האוטומט התאי שכתבנו ויוצרת ממצב רנדומלי פסים בצבעים מתחלפים.
- קובץ essay – הקובץ הזה ☺
- קובץ cell_automata_exercise.exe – קובץ ההרצה ל-Windows (היה קשה לבנות אותו ל-Windows 32 סיביות (כדי שירץ גם ב-32 וגם ב-64 סיביות), שכן שנינו משתמשים במחשבי mac חדשים יחסית).
- תיקיית performance_charts – כוללת קבצי excel של המידע על 17 ההרצות שנציג בהמשך וכן הגרפים שנציג.

קוד המקור:

הקוד עצמו מתועד היטב. מבנה הקוד הוא (קידוד הצבעים: קובץ/מודול, מחלקה, משתנה סטטי, מתודה):

- קובץ main – קובץ ההרצה הראשי, כולל פונקציה יחידה שיוצרת את האוטומט התאי שהתבקש בתרגיל וכן את אובייקט הממשק הגרפי ומתחילה את הממשק הגרפי.
- קובץ requirements.txt – חבילות הפיתון הדרושות לפרויקט: matplotlib (לממשק הגרפי) ו-Numpy (לחישובים מטריציונים, בעבור מטריצת התאים).
- תיקיית automata_target_history – שם בסיום ריצה של הסימולטור, נשמרים הנתונים אודות המרחק ממצב האידיאלי בכל צעד של האוטומט. נשמר כ-csv בעל שתי עמודות (Simulation Step ו-Minimal Target Distance).

- מודול `cell_automata_gui` – מודול המורכב מקובץ יחיד שבונה את הממשק הגרפי. הממשק הגרפי מבוסס `matplotlib` והוא כולל במרכזו מטריצה שגודלה על פי האוטומט התאי. בנוסף, הוא כולל שני כפתורים: `restart` (המאתחל מחדש את האוטומט התאי, כולל רנדום חדש של המטריצה) ו-`stop/start` על מנת לעצור ולהתחיל את ריצת הסימולציה. מעל המטריצה יש שני שדות שמעניינים אותנו: `generation` (שמציג את הצעד הנוכחי בסימולציה) ו-`target_minimal_distance` (שעליו אסביר בהרחבה בהמשך).

- מודול `cell_automata` – מודול שמכיל את הבסיס ליצירת לוגיקות שונות של אוטומטים תאיים. יש בו מספר קבצים:

- `base_cell_2d_automata` – קובץ המגדיר את הבסיס לכל אוטומט תאי במחלקה `BaseCell2DAutomata`. הוא כולל את המתודות הבאות:
 - `__init__` – מקבל רק את גודל המטריצה (אורך ורוחב) ומאתחל מאפייני עזר.
 - `init_grid` – פונקציה אבסטרקטית למימוש על ידי מחלקות יורשות. מטרתו להחזיק את הלוגיקה של אתחול המטריצה, למשל, רנדום המטריצה תוך הקפדה על מספר זהה של תאים שחורים ולבנים (כמו בדרישות התרגיל)
 - `inner_update_grid` – פונקציה אבסטרקטית למימוש על ידי מחלקות יורשות. שם נמצאת לוגיקת עדכון המטריצה בעבור המטריצה.
 - `get_grid_distance` – פונקציה אבסטרקטית בעבור ניתוח מרחק המטריצה הנוכחית ממצב המטריצה או בכללי היוריסטיקה על מצב רצוי כלשהוא או מאפיין מסוים.
 - `update_grid` – פונקציה שבודקת האם יש לעדכן את המטריצה בעוד צעד עדכון על ידי קריאה ל-`inner_update_grid` או שלא (בשל אי אתחול הלוח או בשל מצב סטטי של הלוח), אם יש לעדכן אז הוא קורא לפונקציית העדכון.
 - `is_stable` – מחזיר `True` אם המטריצה הנוכחית שווה למטריצה הקודמת.
 - `get_grid_shape` – מחזיר את צורת המטריצה, כלומר אורכה ורוחבה.
- `stripes_cell_2d_automata` – כולל מימוש של `BaseCell2DAutomata` בשם `BaseCell2DAutomata` שהוא כולל לוגיקה שיוצרת פסים אנכיים מתחלפים שחורים-לבנים ממצב התחלתי של מטריצה מרונדמת לחלוטין עם מספר שווה של תאים שחורים ולבנים.

המחלקה `StripesCell2DAutomata` כוללת את הדברים הבאים:

- משתנה סטטי בשם `_TARGET_CELL_STATES` – כולל את מצב המטריצה של תאי האוטומט. מילון הממפה בין הערך בתא הנוכחי לבין מצב השכנים שלו. בעבור תא עם הערך `True` (שחור), המצב מטריצה היא שהשכן שלו מלמעלה ומלמטה יהיו גם כן שחורים וכל השאר לבנים. בעבור תא עם הערך `False` (לבן), המצב הוא הופכי.

- משתנה סטטי בשם `STABLE_STATES` – כולל רשימת מצבי תא (השכונה שלו) שבעבורם נגדיר שהתא לא ישתנה. נועד לייצוב המטריצה בעבור תאים שמאוד קרובים למטרה. בפועל כולל את כל צורות ה-n של השכונה (כלומר רק תא אחד לא במצב מטרה – נצפה שהוא ישתנה).
- משתנה סטטי בשם `DISTANCE_TEST_INDICES` – כולל רשימת אינדקסים שיגדירו מבחנים מהמצבים האידיאליים של כל שכונת תאים שעל פיהם נגדיר את הערך הבא של התא. יש חשיבות לסדר האיברים ברשימה שכן במקרה של שוויון, אינדקסים ראשונים יקבלו קדימות.
- החלטנו לשים שם את כל קבוצות האינדקסים של תאים צמודים באורך 3 מהשכונה. כלומר התאים העליונים, תחתונים, ימנים, שמאליים, פינת ימין למעלה ושכניה וכנ"ל בעבור כל שאר הפינות (ראה `דיון בלוגיקת האוטומט`).
- `_init` - מקבל רק את גודל המטריצה ומתחיל בלהעביר אותם לאתחול מחלקת האבא (`BaseCell2DAutomata`) ולאחר מכן יוצר מאפיין שכולל את המצבי לוח שהם המטרה שלנו (נשתמש בהם כדי לבדוק כמה אנחנו רחוקים מהמטרה שלנו).
- `init_grid` – מימוש של הפונקציה האבסטרקטית מהאבא, בפועל מאתחל את המטריצה להיות מטריצה שבא בדיוק מחצית מהתאים מרונדמים להיות שחורים. הפונקציה שמרנדמת את תאי המטריצה נמצאת בקובץ `utils`.
- `inner_update_grid` – בעבור כל תא במטריצה קורא לפונקציה `get_cell_next_value` שמחזירה את הערך הבא של תא מסוים. לאחר מכן המתודה משנה את הערך במטריצה להיות הערך שהוחזר. בתחילתה היא שומרת את המטריצה הנוכחית במאפיין שמחזיק במטריצה הקודמת.
- `get_grid_distance` – מימוש של הפונקציה האבסטרקטית מהאבא. כולל פונקציה פנימית בשם `reduce_equal_adjacent_columns` שבהינתן מטריצה מחזירה מטריצה שבה עמודות סמוכות שוות הופכות לאחת (שכן על פי ניסוח התרגיל לא מצוין שעמודה חייבת להיות ברוחב של תא אחד בלבד). בכל מקרה זה משפיע אך ורק על מדד המרחק ממצב היעד.
- לאחר הפעלת הפונקציה הפנימית, משווים את הפלט שלה למטריצות היעד שלנו וסופרים את כל הפעמים שבהם יש הבדל כלשהו ביניהן ולוקחים את סכום ההבדלים המינימלי בהשוואה עם מטריצות היעד (שכן אליה אנחנו קרובים יותר). מתמטית מימשנו מרחק מדרגה ראשונה סטנדרטי אך יותר קל להסתכל על זה כמרחק בין מחרוזות של סכום כל המקומות בהם צריך לשנות את הערך, ששמו גם מרחק לבינשטיין.
- `should_stay_stable` – מחזיר True אם התא ושכניו נמצאים כבר במצב מטרה וכן אם התא ושכניו מתאימים לאחד ממבחני היציבות, שהזכרנו בנקודה השנייה במאפיין הסטטי `STABLE_TESTS`. אחרת מחזיר False.

- `get_cell_next_value` – הפונקציה שבה ממומשת הלוגיקה שאחראית ליצירת הפסים האנכיים המתחלפים ומחברת בין הרבה מהרכיבים שהזכרנו. מתחילה בלהגדיר את השכונה של התא, תוך שמירה על זה שתמיד יהיה בגודל 3 על 3 על ידי wrapping בגבולות. אנו נעזרים בהמשך מספר פעמים במחלקת עזר שיצרנו לחישוב מרחק של "שכונת" תאים ממצב כלשהו בהינתן אינדקסים מסוימים (`TargetDistanceCalculator`).
- לאחר מכן קוראים לפונקציה `should_stay_stable` שהזכרנו בעבר ואם היא מחזירה True נחזיר את הערך הנוכחי. אחרת נשתמש במחלקת העזר לחישוב מרחקים משכונות תאים על מנת לחשב את המרחקים של השכונה הנוכחית מהתאים שמוגדרים במשתנה הסטטי `DISTANCE_TEST_INDICES`. ממרחקים אלה ניקח את המרחק המינימלי מבין מצבי היעד שכל קבוצת תאים הושוותה אליו (שכן זה מתאר את המרחק מהמצב יעד הקרוב יותר). כלומר כרגע יש לנו ביד את המרחק של התא הנוכחי ושכונתו מלהיות במצב יעד בעבור בכל אחת מקבוצות התאים שהזכרנו. ניקח את המרחק המינימלי ונשאף לתקן אותו על ידי בחירת הערך לתא שימנע את המרחק (ראה **דיון בלוגיקת האוטומט**).
- קובץ `consts` – כולל קבועי עזר שימושיים להגדרת קבוצות של אינדקסים רלוונטיים בעבור אוטומטים תאיים. למשל האינדקסים של קבוצת התאים העליונים/ תחתונים/ ימנים/ שמאליים בשכונה. כנ"ל איחודים והפרשים שימושיים של הקבוצות שהוזכרו בעבר ואנו משתמשים בהם רבות כעזר.
- קובץ `utils` – כולל עזרים כלליים לאוטומטים תאיים:
 - `get_balanced_random_grid` – פונקציה שמקבלת גודל של מטריצה ומחזירה מטריצת numpy דו-מימדית שמחצית מערכיה הם שחורים ומיקומם רנדומלי וחצי מערכיה הם לבנים.
 - `TargetDistanceCalculator` – מחלקה שמקבלת מטריצה שמייצגת שכונה של תאים (בגודל 3 על 3) וכן מצבי יעד של השכונה ובמחלקה ממומשות שתי פונקציות:
 - `is_on_target` – משווה מטריציונית האם השכונה נמצאת באחד ממצבי היעד.
 - `get_distance` – מקבלת קבוצת אינדקסים של תאים ומחזירה את המרחק של השכונת תאים ממצבי היעד בעבור האינדקסים הללו.

מדד למרחק האוטומט ממצב המטרה:

המדד שנבחר הוא מרחק לבינשטיין. כלומר, בדומה למחרוזות, בהן הוא מציין את מספר התווים שיש להחליף כדי להגיע לשוויון, במקרה שלנו הוא מייצג את מספר התאים שצריך להחליף את הערך שלהם כדי להגיע לשוויון עם מצב מטרה שלנו. משום שאצלנו יש שני מצבי מטרה, ניקח את המינימלי ביניהם. בכך אנו רואים לרוב התקדמות עקבית ואחידה (ננתח בהמשך עוד) והמדד משתווה לאפס במצב המטרה.

חשוב לשים לב שמשום שבהוראות התרגיל לא מצוין האם עמודה שחורה/לבנה צריכה להיות ברוחב של תא אחד, החשבנו גם עמודות ברוחב של יותר מתא אחד. עם זאת, אין זה נפוץ (כמו שתוכלו לראות בניתוח התוצאות) שהאוטומט שכתבנו פולט מטריצה עם עמודה ברוחב של יותר מתא אחד ובפועל זה קורה בתוחלת של אחת לסביבות העשר הרצות.

דיון בלוגיקת האוטומט:

לאחר סקירה כללית של לוגיקת הקוד שלנו. נציג מעט יותר לעומק את הלוגיקה שאחראית ליצירת הפסים האנכיים ומדוע בחרנו בה, שכן הושקעו בכך שעות רבות של שנינו.

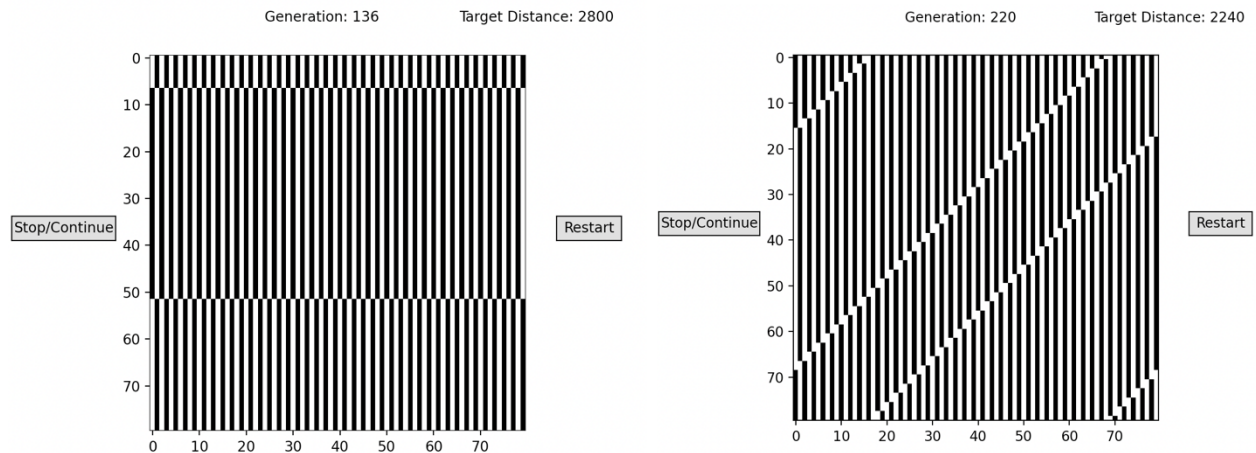
ראשית, רכיב חשוב בלוגיקה זה רכיב "היציבות". בעבור תא שהוא ושכניו ונמצאים כבר במצב מטרה, אין לנו מוטיבציה להחליף את ערכו. גם אם לא כל האוטומט (המטריצה) נמצאת במצב המטרה, בהחלפת הערך, אין תועלת בעבור תנועה לעבר המטרה הגלובאלית ואף תרחיק אותנו בשל חוסר היציבות שהיא תיצור. ערך ההתקדמות לעבר המטרה הגלובאלית של האוטומטית תנבע משינוי התאים שנמצאים בקצוות בין קבוצות תאים שהם ושכניהם נמצאים במצב מטרה כך שהם יתפשטו לכל המטריצה.

כמו כן, חקרנו רבות את רעיון הרנדומיזציה בעבור האוטומט התאי. אינטואיטיבית, זה כלי מאוד חזק שמאפשר לנו באמצעות שינויים הסתברותיים לוקאלית, באמצעות חוק המספרים הגדולים, להגיע לתוצאות גלובאליות. עם זאת, גם אם ההרמטיות שלנו השתפרה (פחות מצבים בהם קשה לנו מאוד להתקדם אל עבר המטרה), מספר האיטרציות הדרושות בעבור התכנסות עלה גם הוא דרמטית. התחלנו מהסתברויות פשוטות בעבור החלפת הערך של התא ולבסוף שילבנו חישובים הסתברותיים בלוגיקות המורכבות יותר שהצגנו.

יתרה מכך, נשים לב שאנו עושים שימוש נרחב במרחק לבינשטיין (שאותו הצגנו בחלק הקודם) לא רק במטריקת המרחק הגלובאלי של האוטומט אלא גם לוקאלית בעבור חישוב הערך הבא של התא. הגדרנו מספר קבוצות מיקומי תאים שמעניינות אותנו (נסביר למה הן מעניינות בפסקה הבאה) ובעבור מרחק הלבינשטיין עליהן. ניקח את הקבוצה בעבורה המרחק לבינשטיין הוא מינימלי (כלומר אנו נמצאים כמה שיותר קרובים למצב מטרה בעבור הכיוון הזה) ועל פיה נבחר את הערך הבא של התא, ככה נתפשט ונתקדם אל עבר מצב המטרה הגלובאלי.

בחירת קבוצות מיקומי התאים נעשתה לאחר הבחנה של התנהגות לא יציבה ושהוללת deadlock רבים כאשר אנו מסתכלים על המרחק של כל שכונת התאים. כאשר נסתכל על כיוון ספציפיים סביב התא, נוכל לשלוט על ההתפשטות ולהימנע באופן כמעט גורף ממצבים כאלה. על כן, (לאחר ניסויים של שעות) בחרנו לקחת את כל מיקומי קבוצות התאים הסמוכות באורך 3 (יש 8 כאלה סביב התא).

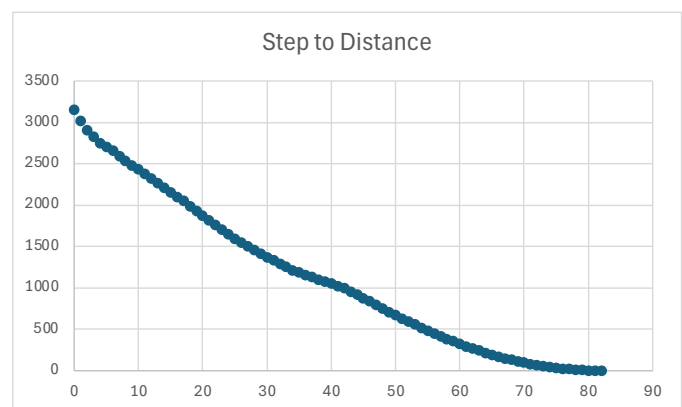
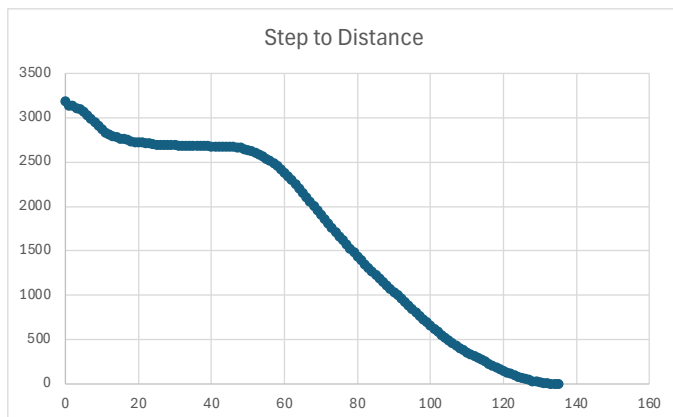
חשוב גם להבחין שיש חשיבות לסדר קבוצות התאים במציאת המרחק המינימלי, שכן במקרה של שוויון, ככל שקבוצות תאים נמצאות קרובה יותר להתחלה היא תקבל עדיפות (תוחזר במינימום). על כן, הקבוצות הראשונות הן קבוצות התאים השמאליים והימניים, הקבוצות האחרונות הן קבוצות התאים העליונים והתחתונים ובאמצע כל קבוצות התאים שמייצגות את הפינות. שכן כך כמעט אין אנו מגיעים למצב באוטומט של עצירה במצבים בעייתיים ולא רצויים. כגון :

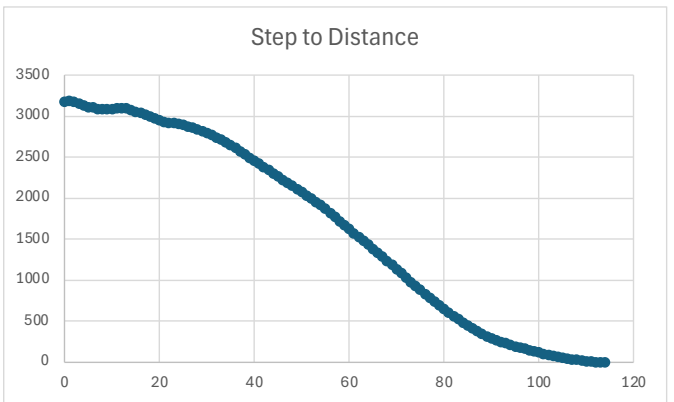
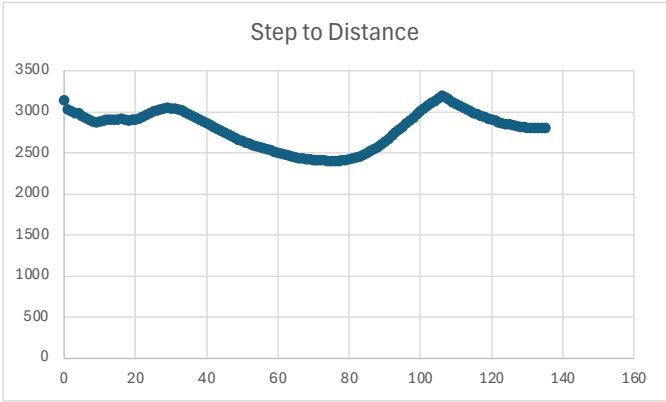
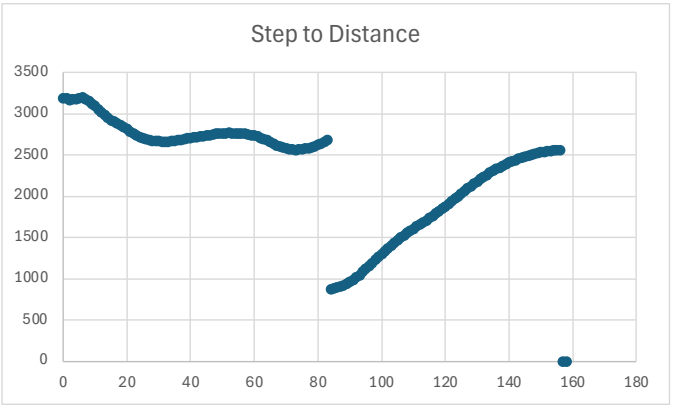
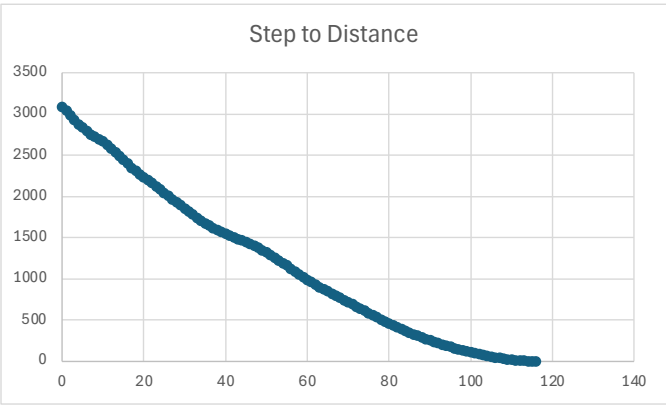
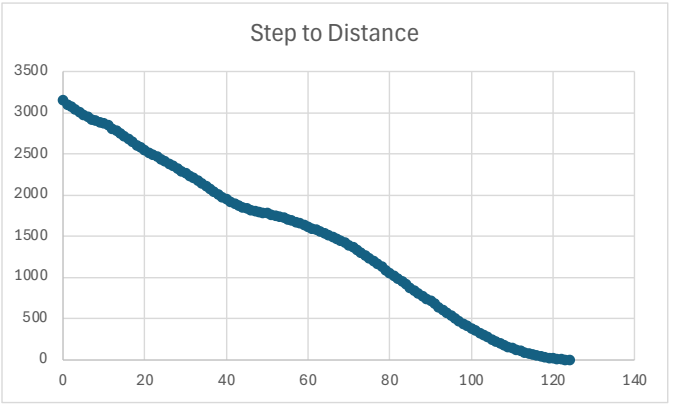
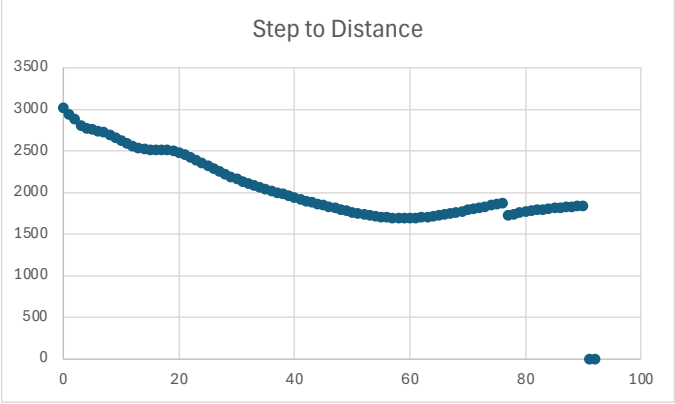


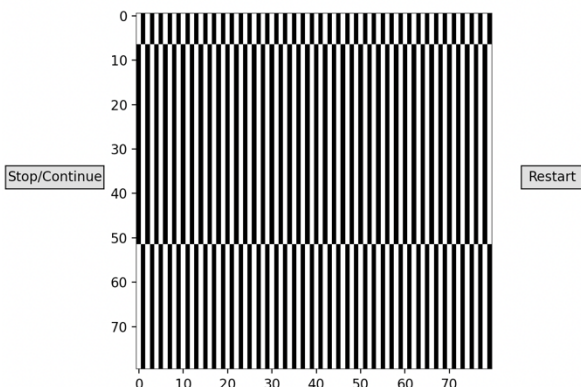
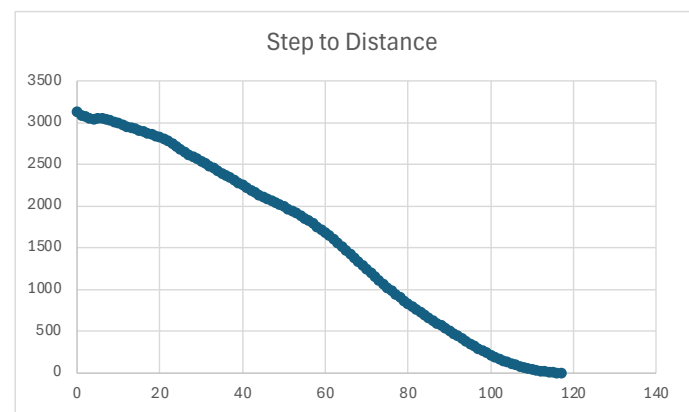
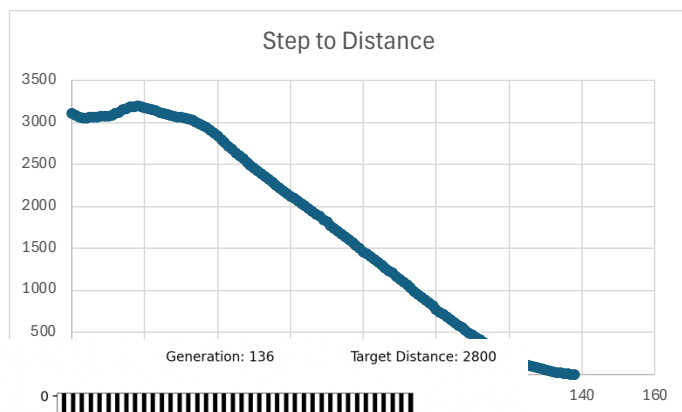
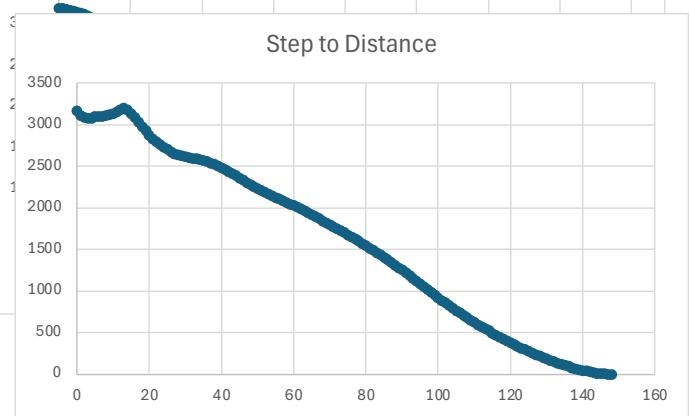
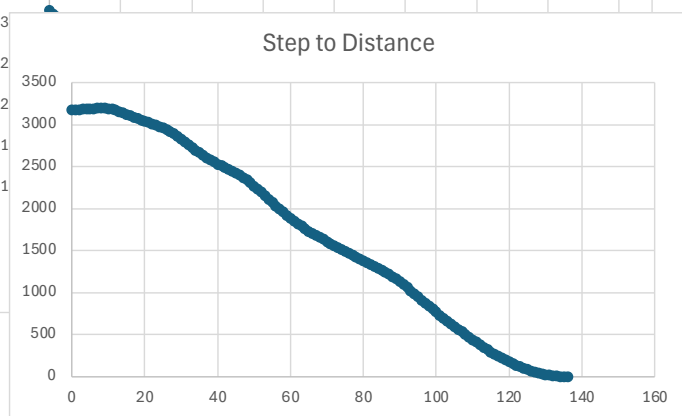
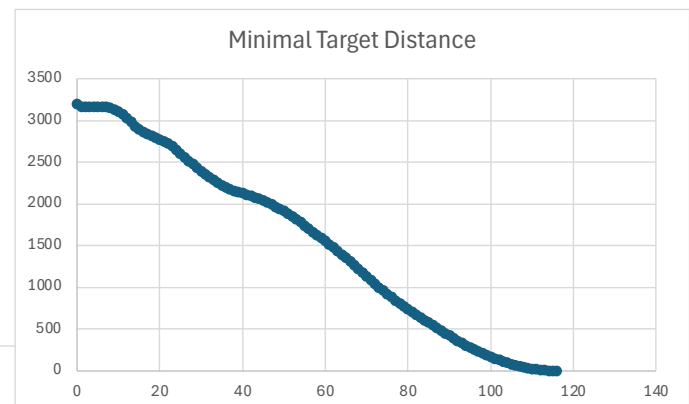
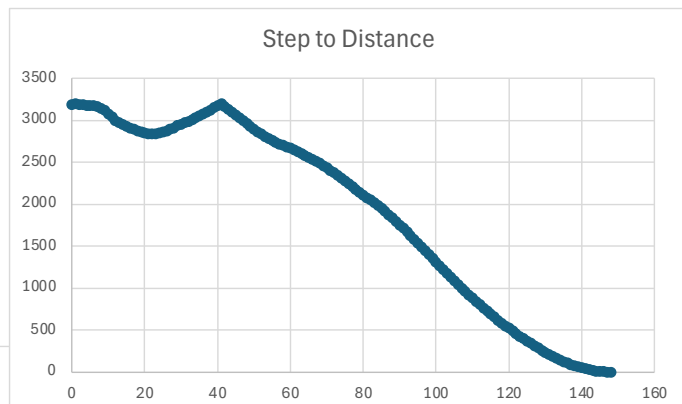
במסגרת ניסויי הרנדומיזציה, בין היתר הגענו לתוצאות סבירות כאשר עשינו בחירה הסתברותית משוקללת בין הכיוונים האפשריים להתקדמות על פי המרחק לבינשטיין שלהם וכן בחירה הסתברותית משוקללת בהינתן הכיוון בעבור הערך על פי המרחק עבור ערך של שחור ושל לבן. השארנו את קטעי הקוד האלה במקומות המתאימים במידה ותרצו לנסות כי זה מעניין ונחמד.

ניתוח תוצאות הרצה:

ההרצה נעשתה על ידי 17 הרצות רצופות של הסימולטור. מתוכן, כל ההרצות מלבד אחת (16 הרצות) הגיעו למצב של מרחק אפס ממצב המטרה של האוטומט על פי המדד שהצגנו. מתוך 16 ההרצות, 2 היו כאלה שבהן היו 2 עמודות ברוחב של 2 תאים. נציג גרפים של התקדמות המדד לאורך צעדי האוטומט (על פי סדר ההרצות):







- בגרף היחיד בו אין המרחק מגיע לאפס זה הגרף המתאים לאוטומט הבא והוא מתאר מצב שבו יש חוסר סנכרון בקו אופקי במקום כלשהו, זה נובע מכך שמצד אחד המרחק בעבור התאים העליונים אומר להם להישאר במצב הנוכחי שלהם (חישוב קל) וכך גם עבור התחתונים. גם בפתרון

ההסתברותי המצב היה דומה וגם אם לעיתים יצא מהפורמט המתואר, הוא לא היה מצליח להתכנס לאורך כל הרוחב. נובע מהתנהגות ה-wrapping.

- בעבור 2 הגרפים בהם יש עמודות ברוחב של יותר מתא אחד (ניתן לזהות אותם על פי 2 קפיצות באמצע הגרף). הסיבה להתנהגות הזאת של הגרף נובעת מכך שכאשר העמודה מתכנסת היא נחשבת עבור המדד כעמודה ולידית אך עד אז היא יוצרת חוסר סנכרון קדימה לכל הרוחב של המטריצה ולכן המדד מציג מספר מאוד גדול. ההתנהגות הזאת נובעת מה-wrapping, בעבור "סגירת מעגל אנכית" ואז שני צידי הקו מנסים להתכנס לעמודה יחדיו אך אין הם יכולים להתפצל לשניים שונים כי יש "סגירת מעגל אנכית" ולכן תמיד יש 2 עמודות ברוחב 2 במקרה הזה.

- בעבור הגרפים עם ההתנהגות הרצויה (הרוב המוחלט), הגרף נראה כמצופה 😊

הערות

תודה רבה על האישור דחייה של הגשת התרגיל, שנינו חיילים בשירות שמושפע מאוד מהמלחמה כרגע וזה עזר לנו מאוד.

בין היתר חשוב לשנינו מאוד לעשות את כל התרגילים ברמה הכי גבוהה כדי לזקק מהם כמה שיותר ידע ולמידה וככה גם ללמוד בהתאם כמה שיותר מהקורס.